Kernel Methods

Machine Learning – Foundations and Algorithms

Prof. Gerhard Neumann
Autonomous Learning Robots
KIT, Institut für Anthropomatik und Robotik

Autonomous Learning Robots Lab
Karlsruher Institut für Technologie

The “thing” about features

Good feature representations are typically very hard to find...

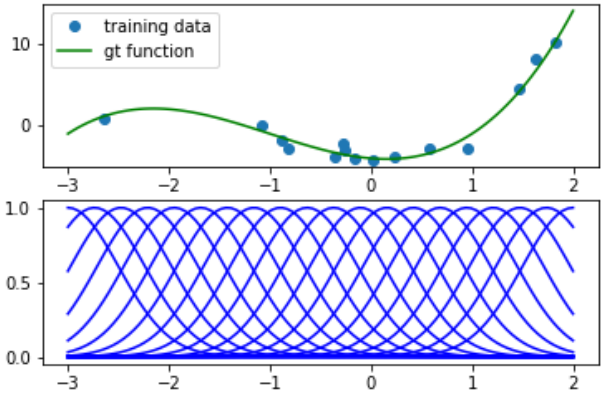
- **Polynomial features:**
  - Number of features increases with O(d^k), where d is dimensionality of data and k is degree of polynom
  - Can get very high feature value differences for large k

- **Radial basis function features:**
  - Uniformly distribute Gaussian basis functions over the input space

$$\boldsymbol{\phi}(\boldsymbol{x}) = [\phi_{\boldsymbol{\mu}_1}(\boldsymbol{x}), \ldots, \phi_{\boldsymbol{\mu}_M}(\boldsymbol{x})]^T, \quad \phi_{\boldsymbol{\mu}_i}(\boldsymbol{x}) = \exp\left(-\frac{||\boldsymbol{x} - \boldsymbol{\mu}_i||^2}{2\sigma^2}\right)$$

$\boldsymbol{\mu}_i \ldots$ i^{th} center location (fixed) $\quad \sigma^2 \ldots$ bandwith

  - Number of basis functions scales exponentially with d
  - Many basis functions located at irrelevant positions with no data



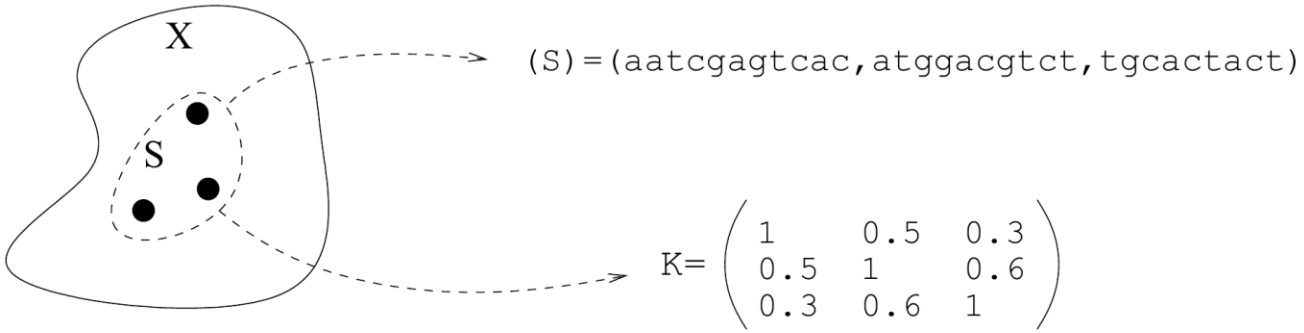
Kernels - Motivation

Can we find a more powerful representation for linear models?

- Yes! **Kernels!**
- They are more generic, scale better and adapt to the data set
- They still preserve simple linear computations

What is a kernel?

Representation by point-wise comparisons



- Define a "comparison function" $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$
- Represent a set of points $\mathcal{S} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$ by **the n x n matrix** $[\boldsymbol{K}]_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$

Kernel Matrix (aka Gram Matrix)

Properties:

- $\boldsymbol{K}$ is always an n × n matrix, whatever the nature of data: the same algorithm will work for any type of data (vectors, strings, ...).
- Total modularity between the choice of function k and the choice of the algorithm.
- Poor scalability with respect to the dataset size (n^2 to compute and store $\boldsymbol{K}$)...
- We will restrict ourselves to a particular class of pairwise comparison functions.

Positive definite kernels

A **positive definite kernel function** *k* is a function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ that is:

(i) Symmetric: $\forall \boldsymbol{x}, \boldsymbol{x}' : k(\boldsymbol{x}, \boldsymbol{x}') = k(\boldsymbol{x}', \boldsymbol{x})$

(ii) Similarity matrix is always positive semi-definite (psd)

$$\boldsymbol{a}^T \boldsymbol{K} \boldsymbol{a} = \sum_{i=1}^{n} \sum_{j=1}^{n} a_i a_j k(\boldsymbol{x}_i, \boldsymbol{x}_j) \geq 0, \quad \forall \boldsymbol{a}, \forall S = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n\}$$

- **Condition for psd matrices**: all eigenvalues >= 0

Kernel methods are algorithms that take such matrices as input.

Example: Linear kernel

The linear kernel is the simplest kernel for vectors

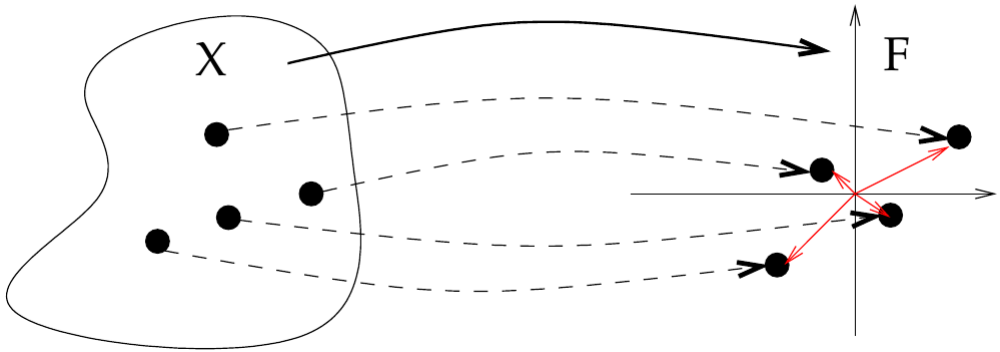
- Its defined by the scalar product:

$$k(\boldsymbol{x}, \boldsymbol{x}') = \langle \boldsymbol{x}, \boldsymbol{x}' \rangle, \quad \text{where } \langle \cdot, \cdot \rangle \text{ denotes the inner product}$$

- It is always positive definite:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} a_i a_j \langle \boldsymbol{x}_i, \boldsymbol{x}_j \rangle = \Big\| \sum_i a_i \boldsymbol{x}_i \Big\|^2 \geq 0$$

Kernels in Feature Spaces



Let $\phi : \mathcal{X} \to \mathbb{R}^d$ be an arbitrary feature function, then $k(\boldsymbol{x}, \boldsymbol{x}') = \langle \boldsymbol{\phi}(\boldsymbol{x}), \boldsymbol{\phi}(\boldsymbol{x}') \rangle$ defines a positive definite kernel.

Proof: $$\sum_{i=1}^{n} \sum_{j=1}^{n} a_i a_j \langle \boldsymbol{\phi}(\boldsymbol{x}_i), \boldsymbol{\phi}(\boldsymbol{x}_j) \rangle = \Big\| \sum_{i} a_i \boldsymbol{\phi}(\boldsymbol{x}_i) \Big\|^2 \geq 0$$

Kernels as inner products

Theorem (Aransjan 1950):

k is a positive semi-definite kernel on the set $\mathcal{X}$ if and only if there exists a feature space $\mathcal{H}$ and a feature mapping

$$\phi : \mathcal{X} \rightarrow \mathcal{H}$$

such that for any $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{X}$:

$$k(\boldsymbol{x}, \boldsymbol{x}') = \left\langle \phi(\boldsymbol{x}), \phi(\boldsymbol{x}') \right\rangle$$

- **Every psd kernel comes with an associated feature space!**

Kernel Regression

- What is a kernel?
- Examples for kernels
- The kernel trick
- Kernel Ridge Regression

Example: polynomial kernel

For $\boldsymbol{x} = [x_1, x_2]^T$, let $\boldsymbol{\phi}(\boldsymbol{x}) = [x_1^2, \sqrt{2}x_1x_2, x_2^2]$

The kernel is defined by:

$$\begin{aligned} k(\boldsymbol{x}, \boldsymbol{x}') &= x_1^2 x_1'^2 + 2x_1x_2x_1'x_2' + x_2^2 x_2'^2 \\ &= (x_1x_1' + x_2x_2')^2 \\ &= \langle \boldsymbol{x}, \boldsymbol{x}' \rangle^2 \end{aligned}$$

Kernel for polynomials of degree d:

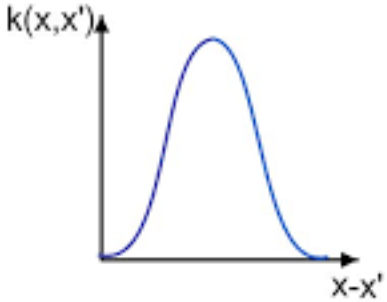
$$k(\boldsymbol{x}, \boldsymbol{x}') = \langle \boldsymbol{x}, \boldsymbol{x}' \rangle^d$$

Example: Gaussian Kernel

The Gaussian kernel is defined by:

$$k(\boldsymbol{x}, \boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{y}\|^2}{2\sigma^2}\right)$$

- where σ is the bandwidth parameter



Often also called:

- Radial basis function kernel (RBF)
- Squared exponential kernel

It is the most used kernel for kernel methods

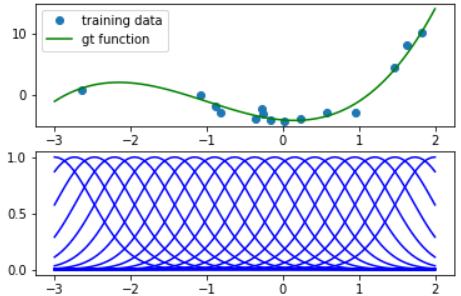
Is the Gaussian kernel a valid p.d. kernel?

Yes! It can be shown that the kernel is a valid product of two **infinite dimensional feature vectors**

- Consider the following feature function:

$$\phi_{\boldsymbol{\mu}}(\boldsymbol{x}) = 1/Z \exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{\mu}\|^2}{\sigma^2}\right), \quad \forall \boldsymbol{\mu} \in \mathbb{R}^d$$

- I.e. we have an infinite amount of features (for every possible center $\boldsymbol{\mu}$)
- Z is a normalization constant (which we will ignore)



The Gaussian kernel would put a RBF center at every possible location

The Gaussian kernel can be computed as (not shown)**:**

$$k(\boldsymbol{x},\boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|^2}{2\sigma^2}\right) = \langle \phi_{\boldsymbol{\mu}}(\boldsymbol{x}), \phi_{\boldsymbol{\mu}}(\boldsymbol{y}) \rangle = \int \phi_{\boldsymbol{\mu}}(\boldsymbol{x})\phi_{\boldsymbol{\mu}}(\boldsymbol{y}) d\boldsymbol{\mu}$$

- Inner product becomes an integral due to infinite amount of dimensions
- We never have to represent these vectors if we can directly compute their inner products

Kernel Regression

- What is a kernel?
- Examples for kernels
- The kernel trick
- Kernel Ridge Regression

Kernel Trick

So why do we do this?

- Kernels can be used for most feature based algorithms that can be rewritten such that they contain inner products of feature vectors
  - This is true for almost all feature based algorithms (Linear regression, Support Vector Machines, …)
  - This is called the Kernel Trick
- Kernels can be used to map the data x in an infinite dimensional feature space (i.e., a function space)
  - The feature vector never has to be represented explicitly
  - As long as we can evaluate the inner product of two feature vectors
- Hence, we obtain a more powerful representation than standard linear feature models

A few kernel identities

Let $\boldsymbol{\Phi}_X = \begin{bmatrix} \boldsymbol{\phi}(\boldsymbol{x}_1)^T \\ \vdots \\ \boldsymbol{\phi}(\boldsymbol{x}_N)^T \end{bmatrix} \in \mathbb{R}^{N \times d}$ then the following identities hold:

- **Kernel matrix:** $\boldsymbol{K} = \boldsymbol{\Phi}_X \boldsymbol{\Phi}_X^T$
  - Check: $[\boldsymbol{K}]_{ij} = \boldsymbol{\phi}(\boldsymbol{x}_i)^T \boldsymbol{\phi}(\boldsymbol{x}_j) = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$
- **Kernel vector:** $\boldsymbol{k}(\boldsymbol{x}^*) = \begin{bmatrix} k(\boldsymbol{x}_1, \boldsymbol{x}^*) \\ \vdots \\ k(\boldsymbol{x}_N, \boldsymbol{x}^*) \end{bmatrix} = \begin{bmatrix} \boldsymbol{\phi}(\boldsymbol{x}_1)^T \boldsymbol{\phi}(\boldsymbol{x}^*) \\ \vdots \\ \phi(\boldsymbol{x}_N)^T \boldsymbol{\phi}(\boldsymbol{x}^*) \end{bmatrix} = \boldsymbol{\Phi}_X \boldsymbol{\phi}(\boldsymbol{x}^*)$

Kernel trick

Recap: Ridge Regression

- Squared error function + L2 regularization
- Linear feature space
- Not directly applicable in infinite dimensional feature spaces

Objective:

$$L_{\text{ridge}} = \underbrace{(\boldsymbol{y} - \boldsymbol{\Phi w})^T (\boldsymbol{y} - \boldsymbol{\Phi w})}_{\text{sum of squared errors}} + \lambda \underbrace{\boldsymbol{w}^T \boldsymbol{w}}_{L_2 \text{ regularization}}$$

Solution:

$$\boldsymbol{w}^*_{\text{ridge}} = \underbrace{(\boldsymbol{\Phi}^T \boldsymbol{\Phi} + \lambda \boldsymbol{I})^{-1}}_{d \times d \text{ matrix inversion}} \boldsymbol{\Phi}^T \boldsymbol{y}$$

Matrix inversion infeasible in infinite dimensions

Kernel trick

We can apply the "kernel trick":

- Rewrite solution as inner products of the feature space!
- We can do this by using the following matrix identity

$$(\lambda \boldsymbol{I} + \boldsymbol{AB})^{-1}\boldsymbol{A} = \boldsymbol{A}(\lambda \boldsymbol{I} + \boldsymbol{BA})^{-1}$$

  - "Searle set of identities", The Matrix Cookbook

$$\boldsymbol{w}^* = \underbrace{(\boldsymbol{\Phi}^T\boldsymbol{\Phi} + \lambda \boldsymbol{I})^{-1}}_{d \times d \text{ matrix inversion}} \boldsymbol{\Phi}^T \boldsymbol{y} = \boldsymbol{\Phi}^T \underbrace{(\boldsymbol{\Phi}\boldsymbol{\Phi}^T + \lambda \boldsymbol{I})^{-1}}_{N \times N \text{ matrix inversion}} \boldsymbol{y}$$

  - With $\boldsymbol{A} = \boldsymbol{\Phi}^T$ and $\boldsymbol{B} = \boldsymbol{\Phi}$

The „tale of the two Pseudo Inverse“

- **Two side inverse:** $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}, \quad \boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$
  - Standard matrix inversion… does often not exist
- **Left-Pseudo Inverse (see linear regression):**

$$\boldsymbol{A}^{\dagger}_{\text{left}} = (\boldsymbol{A}^T\boldsymbol{A})^{-1}\boldsymbol{A}^T, \quad \boldsymbol{A}^{\dagger}_{\text{left}}\boldsymbol{A} = (\boldsymbol{A}^T\boldsymbol{A})^{-1}\boldsymbol{A}^T\boldsymbol{A} = \boldsymbol{I}$$

- **Right-Pseudo Inverse:**

$$\boldsymbol{A}^{\dagger}_{\text{right}} = \boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1}, \quad \boldsymbol{A}\boldsymbol{A}^{\dagger}_{\text{right}} = \boldsymbol{A}\boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1} = \boldsymbol{I}$$

Difference: Size of the matrix that needs to be inverted

- For regularized version, both are mathematically (but not numerically) equivalent

$$(\boldsymbol{A}^T\boldsymbol{A} + \lambda\boldsymbol{I})^{-1}\boldsymbol{A}^T = \boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T + \lambda\boldsymbol{I})^{-1}$$

Kernel Regression

- What is a kernel?
- Examples for kernels
- The kernel trick
- Kernel Ridge Regression

Kernel ridge regression

The "kernelized" solution is given by:

$$\boldsymbol{w}^* = \boldsymbol{\Phi}^T \underbrace{(\boldsymbol{\Phi}\boldsymbol{\Phi}^T + \lambda \boldsymbol{I})^{-1}}_{N \times N \text{ matrix inversion}} \boldsymbol{y} = \boldsymbol{\Phi}^T \underbrace{(\boldsymbol{K} + \lambda \boldsymbol{I})^{-1}\boldsymbol{y}}_{\boldsymbol{\alpha}} = \boldsymbol{\Phi}^T \boldsymbol{\alpha}$$

- Instead of inverting a d x d matrix, we can now invert an N x N matrix
- Is beneficial for d >> N (e.g., d is infinite)
- Still, $\boldsymbol{w}^* \in \mathbb{R}^d$ is potentially infinite dimensional and can not be represented

Yet, we can evaluate the function f that is specified by $\boldsymbol{w}^*$:

$$f(\boldsymbol{x}) = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{w}^* = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{\Phi}^T \boldsymbol{\alpha} = \boldsymbol{k}(\boldsymbol{x})^T \boldsymbol{\alpha} = \sum_i \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x})$$

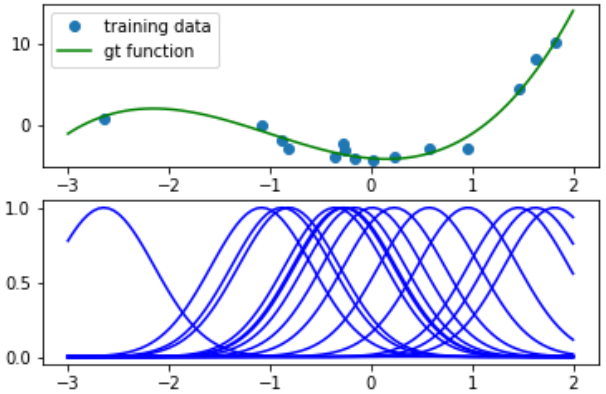
- The function is evaluated by a weighted sum of kernel activations evaluated at the training data

Examples and comparison to RBF regression

For a Gaussian kernel, the prediction corresponds to

$$f(\boldsymbol{x}) = \sum_i \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}) = \sum_i \alpha_i \exp\left(-\frac{||\boldsymbol{x} - \boldsymbol{x}_i||^2}{2\sigma^2}\right)$$

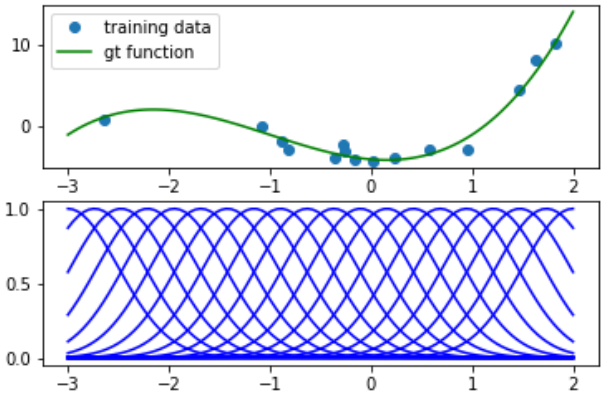
- The kernel allows setting the centres adaptively to the available data!
- One centre per data-point



Comparison: Linear regression with radial basis function (RBF) features

$$f(\boldsymbol{x}) = \sum_i w_i \phi_i(\boldsymbol{x}) = \sum_i w_i \exp\left(-\frac{||\boldsymbol{x} - \boldsymbol{\mu}_i||^2}{2\sigma^2}\right)$$

$\boldsymbol{\mu}_i \ldots$ i^{th} center location (fixed)



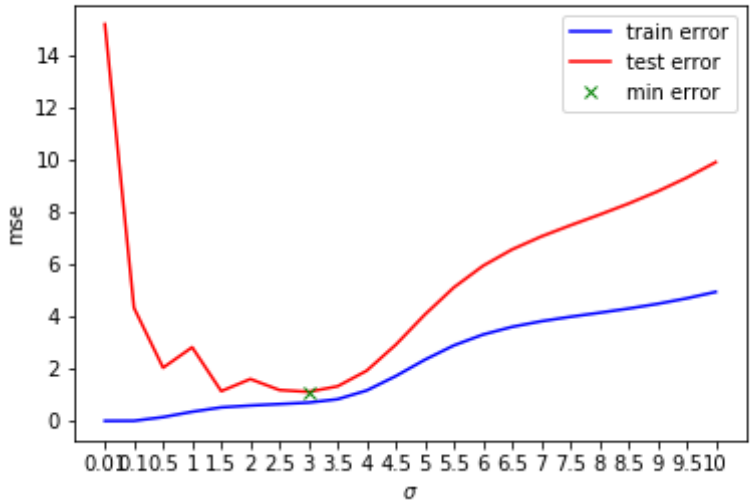
Selecting the hyper-parameters

- The parameters of the kernel, e.g., sigma in

$$k(\boldsymbol{x}, \boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{y}\|^2}{2\sigma^2}\right)$$

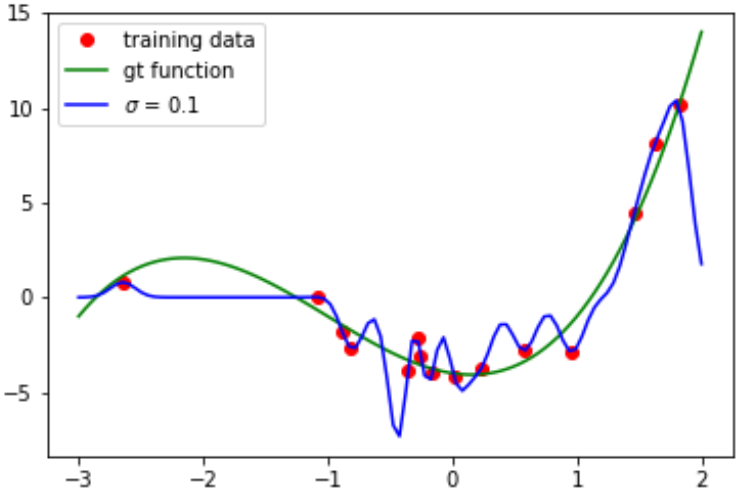
  are called hyper-parameters.

- Choosing them is a model-selection problem (we need a test set).

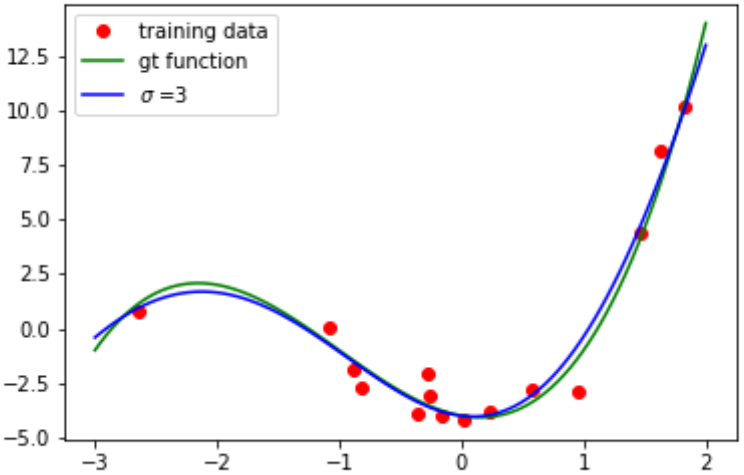


Different bandwidth factors

Overfitting



Good fit



Comparison

- **Ridge Regression**

$$f(\boldsymbol{x}) = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{w}^* = \boldsymbol{\phi}(\boldsymbol{x})^T \underbrace{(\boldsymbol{\Phi}^T \boldsymbol{\Phi} + \lambda \boldsymbol{I})^{-1}}_{d \times d \text{ matrix inversion}} \boldsymbol{\Phi}^T \boldsymbol{y} = \sum_{i=1}^{d} w_i \phi_i(\boldsymbol{x})$$

  - Use if d < N

- **Kernel Ridge Regression**

$$f(\boldsymbol{x}) = \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{w}^* = \overbrace{\boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{\Phi}}^{\boldsymbol{k}(\boldsymbol{x})} \underbrace{(\overbrace{\boldsymbol{\Phi} \boldsymbol{\Phi}^T}^{\boldsymbol{K}} + \lambda \boldsymbol{I})^{-1}}_{N \times N \text{ matrix inversion}} \boldsymbol{y} = \sum_{i=1}^{N} \alpha_i k(\boldsymbol{x}, \boldsymbol{x}_i)$$

  - Use if N < d

Summary: Kernel ridge regression

The solution for kernel ridge regression is given by

$$f^*(\boldsymbol{x}) = \boldsymbol{k}(\boldsymbol{x})^T(\boldsymbol{K} + \lambda \boldsymbol{I})^{-1}\boldsymbol{y}$$

- No evaluations of the feature vectors needed
- Only pair-wise scalar products (evaluated by the kernel)
- Need to invert a N x N matrix (can be costly

Note:

- We have to store all samples in kernel-based methods (they also belong to the instance-based or non-parametric methods)
  - Computationally expensive (matrix inverse is $O(n^{2.376})$) !
- Hyper-parameters of the method are given by the kernel-parameters
  - Can be optimized on validation-set
- Very flexible function representation, only few hyper-parameters

SVMs with Kernels

Support Vector Machines (continued…)

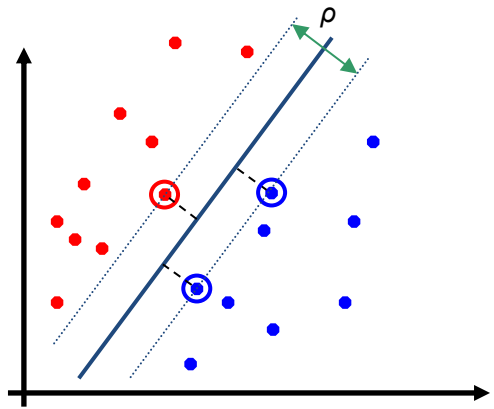
SVMs with features:

- Maximum margin principle
- Slack variables allow for margin violation

$$\begin{aligned} \text{argmin}_{\mathbf{w},\boldsymbol{\xi}} \quad & ||\mathbf{w}||^2 + C\sum_i^N \xi_i, \\ \text{s.t.} \quad & y_i(\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

- Simpler formulation without slack variables

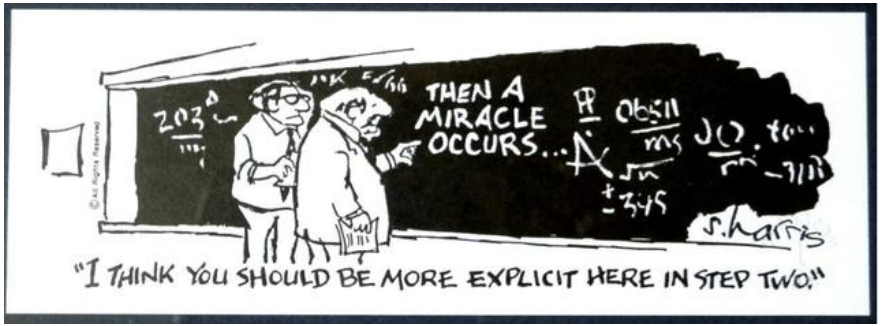
$$\begin{aligned} \text{argmin}_{\mathbf{w}} \quad & ||\mathbf{w}||^2, \\ \text{s.t.} \quad & y_i(\mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 \end{aligned}$$



In order to apply the kernel trick, we need to apply Constrained Optimization!

- … however, we do not have time for going into that
- See RL lecture

Constraint Optimization in SVMs



In constraint optimization, we have to optimize the dual function:

$$\boldsymbol{\lambda}^* = \text{argmax}_{\boldsymbol{\lambda}} g(\boldsymbol{\lambda}) = \text{argmax}_{\boldsymbol{\lambda}} \sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \boldsymbol{\phi}(\boldsymbol{x}_i)^T \boldsymbol{\phi}(\boldsymbol{x}_j), \text{ s.t. } \lambda_i > 0$$

- Where λ_i are Lagrangian multipliers
- The weight vector can then be obtained by

$$\boldsymbol{w}^* = \sum_i \lambda_i y_i \boldsymbol{\phi}(\boldsymbol{x}_i)$$

We can directly use the kernel trick here

$$g(\boldsymbol{\lambda}) = \sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \boldsymbol{k}(\boldsymbol{x}_i, \boldsymbol{x}_j)$$

- Scalar products of the feature vectors can be written as kernels, i.e. $k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{\phi}(\boldsymbol{x}_i)^T \boldsymbol{\phi}(\boldsymbol{x}_j)$

Relaxed constraints with slack

Primal optimization problem:

$$\begin{aligned} \text{argmin}_{\mathbf{w},\boldsymbol{\xi}} \quad & ||\mathbf{w}||^2 + C\sum_{i}^{N}\xi_i, \\ \text{s.t.} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

Dual optimization problem:

$$\max_{\boldsymbol{\lambda}} \sum_i \lambda_i - \frac{1}{2}\sum_i\sum_j \lambda_i\lambda_j y_i y_j \boldsymbol{k}(\boldsymbol{x}_i, \boldsymbol{x}_j)$$

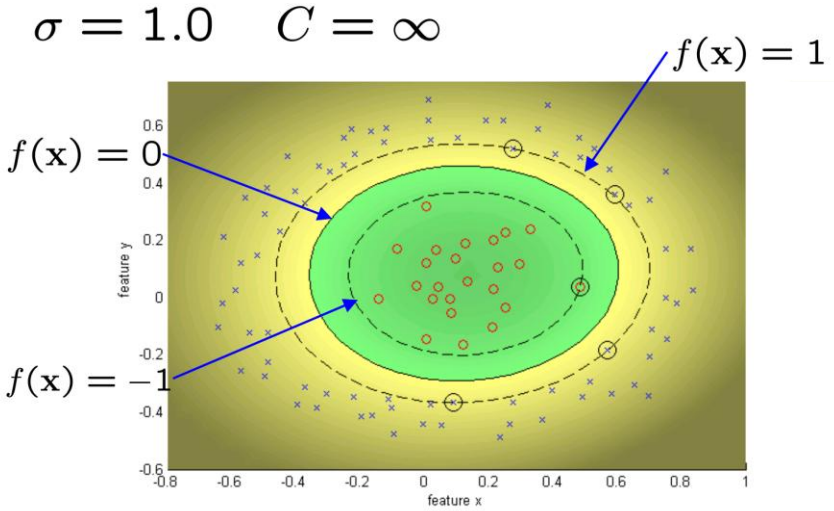
$$\text{s.t. } C \geq \lambda_i \geq 0, \forall i \in [1 \ldots N], \quad \sum_i \lambda_i y_i = 0$$

This is a quadratic program

- Quadratic objective, linear constraints
- Efficient solution solvers exists

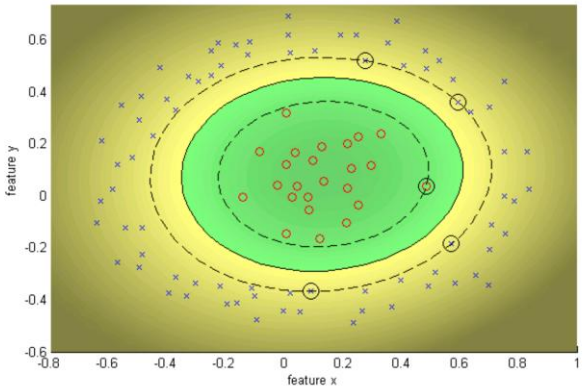
Example: SVM with RBF kernel

Data is non-linearly separable in original space

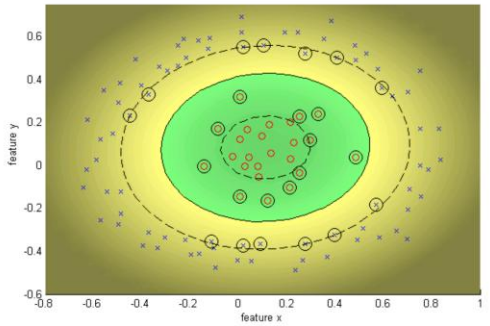


Example: Different Cs

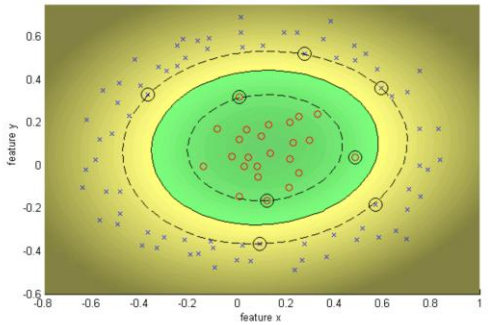
$\sigma = 1.0 \quad C = \infty$



$\sigma = 1.0 \quad C = 10$

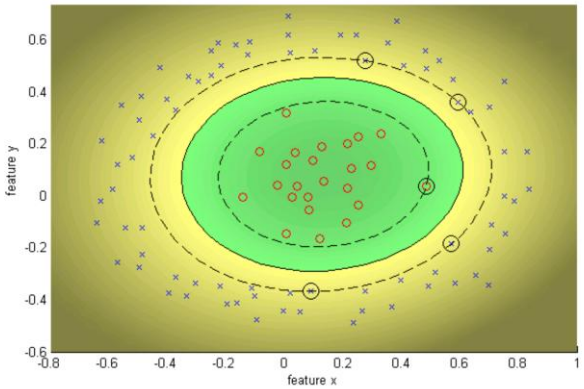


$\sigma = 1.0 \quad C = 100$

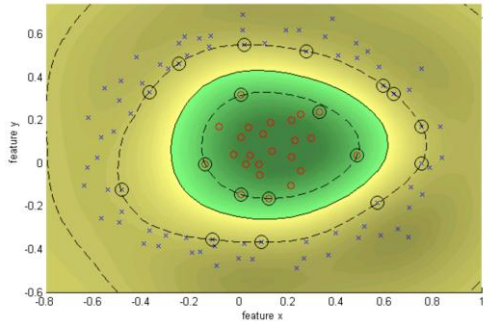


Example: Different sigma

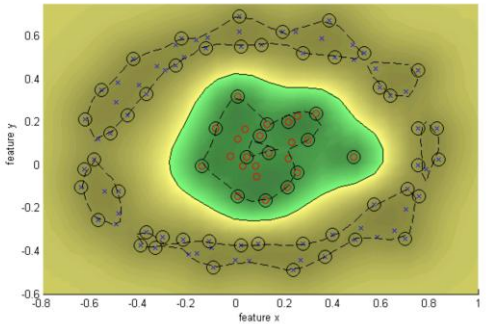
$\sigma = 1.0 \quad C = \infty$



$\sigma = 0.25 \quad C = \infty$



$\sigma = 0.1 \quad C = \infty$



Overfitting

Huge feature space with kernels: should we worry about overfitting?

- SVM objective seeks a solution with large margin
- Theory says that large margin leads to good generalization
- But everything overfits sometimes!!!

Can control overfitting by:

- Setting C (low C -> smaller Complexity)
- Choosing a better Kernel
- Varying parameters of the Kernel (width of Gaussian, etc.)

} Model Selection Problems

Handwritten Digit Clasification

US postal service database

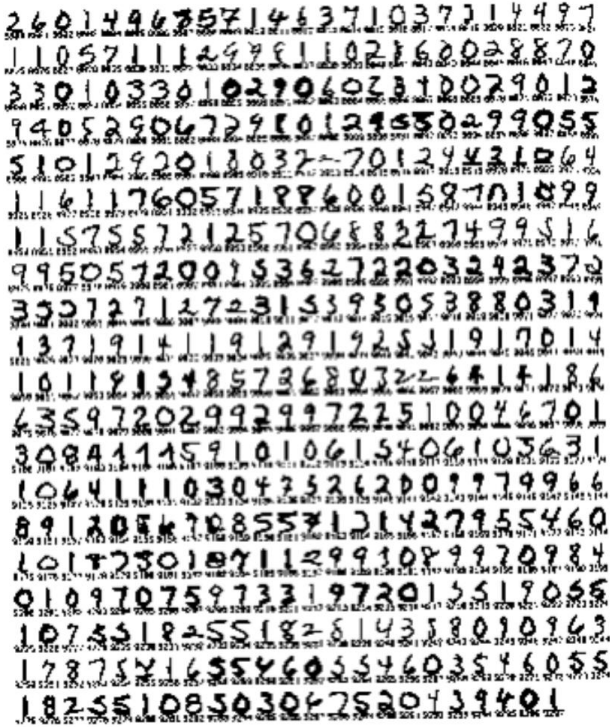
- Human performance: 2.5% error

Various learning algorithms (pre-deep learning

- 16.2%: Decision tree (C4.5)
- 5.9%: 2-layer neural network
- 5.1%: LeNet 1 - 5-layer neural network

Various SVM results

- 4.0%: Polynomial kernel (274 support vectors)
- 4.1%: Gaussian kernel



Handwritten Digit Clasification

Very little overfitting due to max-margin

degree of polynomial	dimensionality of feature space	support vectors	raw error
1	256	282	8.9
2	≈ 33000	227	4.7
3	$\approx 1 \times 10^6$	274	4.0
4	$\approx 1 \times 10^9$	321	4.2
5	$\approx 1 \times 10^{12}$	374	4.3
6	$\approx 1 \times 10^{14}$	377	4.5
7	$\approx 1 \times 10^{16}$	422	4.5

Recent results

- With more training data, better modeling of invariances, etc.
- Error down to about 0.5% with SVMs and 0.4% with neural networks

Takeaway messages

What have we learned today?

- Kernels estimate the similarity between samples
- They represent an inner product in a feature space
  - Allows to use potentially infinite dimensional
  - That's ok due to the kernel trick and regularization
- Many standard ML algorithms can be "kernelized"
  - I.e. rewritten in terms of inner products
  - **Regression:** Kernel Ridge regression, Gaussian Processes (to be covered), Support Vector Regression (not covered)
  - **Classification:** SVMs, Kernel Logistic Regression (not covered)

✓ Very flexible representation that adapts to the complexity of the data

✓ Works well with small data sets

× Hard to scale to more complex (high-D) problems or large datasets